



Cryptography Lab

Matteo Loporchio
28/4/2026
matteo.loporchio@di.unipi.it





UNIVERSITÀ
DI PISA



Lab materials

Scan me!



All materials are available at: <https://github.com/mloporchio/Crypto-Lab-IRPET>

Tools



- Python (<https://www.python.org>)
- Pycryptodome (<https://www.pycryptodome.org>)
- Jupyter (<https://jupyter.org>)
- Google Colab (<https://colab.research.google.com>)



Installing

To install, run:

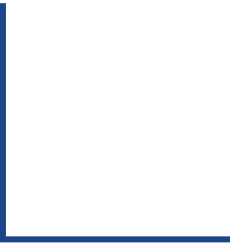
```
> pip3 install pycryptodome jupyter
```

Install and run from a *virtual environment*:

```
> python3 -m venv .venv  
> source .venv/bin/activate  
> pip3 install pycryptodome jupyter  
> ...  
> deactivate
```



Authenticated Data Structures



Authenticated Data Structures

Let C be a collection of generic data objects (e.g., files).

An **Authenticated Data Structure (ADS)** for C incorporates cryptographic information so that:

- 1) an untrusted **prover** can carry out operations on C ;
- 2) a **verifier** can efficiently check the **correctness** of the result of such operations.

Authenticated Data Structures

The prover needs to send to the verifier a **proof of correctness**.

This piece of information allows the verifier to be convinced that the operations on the data structure were carried out correctly.

Authenticated Data Structures

Notice the **asymmetry**:

The **prover** knows the entire content of C (i.e., it explicitly stores the data objects).

The **verifier** knows only a short *digest* of the data structure, but this is sufficient to verify proofs.

Authenticated Data Structures

Correctness: the prover cannot lie about the obtained result (e.g., by sending to the verifier a different value).

Correctness definition depends on the type of data and queries sent by the verifier.

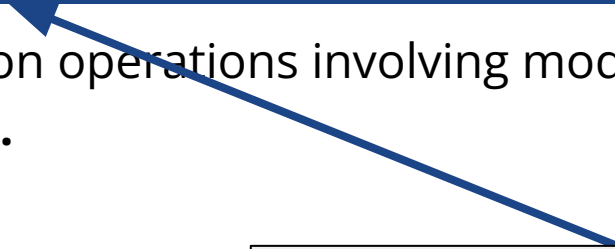
Authenticated Data Structures

Classification based on the cryptographic mechanisms adopted.

- 1) **Hash-based:** rely on cryptographic hash functions.
- 2) **Arithmetic-based:** rely on operations involving modular arithmetic/elliptic curves.

Authenticated Data Structures

Classification based on the cryptographic mechanisms adopted.

- 1) **Hash-based:** rely on cryptographic hash functions.
 - 2) **Arithmetic-based:** rely on operations involving modular arithmetic/elliptic curves.
- 

Today we focus on hash-based data structures.

Cryptographic hash functions

$H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ maps bit sequences of arbitrary length to fixed-size *digests*.

1. For all $x \in \{0, 1\}^*$ it is computationally easy to calculate $y = H(x)$.
2. **Preimage resistance.** For all $y \in \{0, 1\}^n$ it is computationally infeasible to find $x \in \{0, 1\}^*$ such that $H(x) = y$.
3. **Collision resistance.** It is computationally infeasible to find two elements $x_1, x_2 \in \{0, 1\}^*$ such that $H(x_1) = H(x_2)$.

Merkle trees

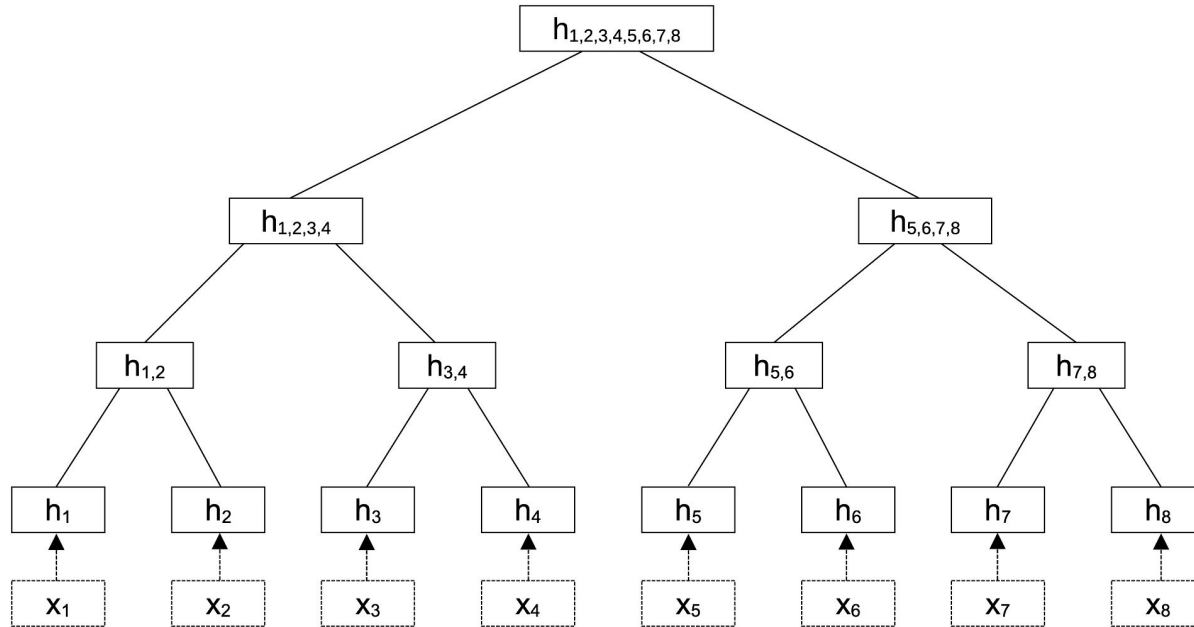
Merkle tree

An authenticated data structure that allows users to:

- 1) verify the integrity of a set of data objects $S = \{x_1, \dots, x_n\}$;
- 2) generate and verify **membership proofs** for such objects.

Membership proof: a cryptographic proof certifying that an element x belongs to S .

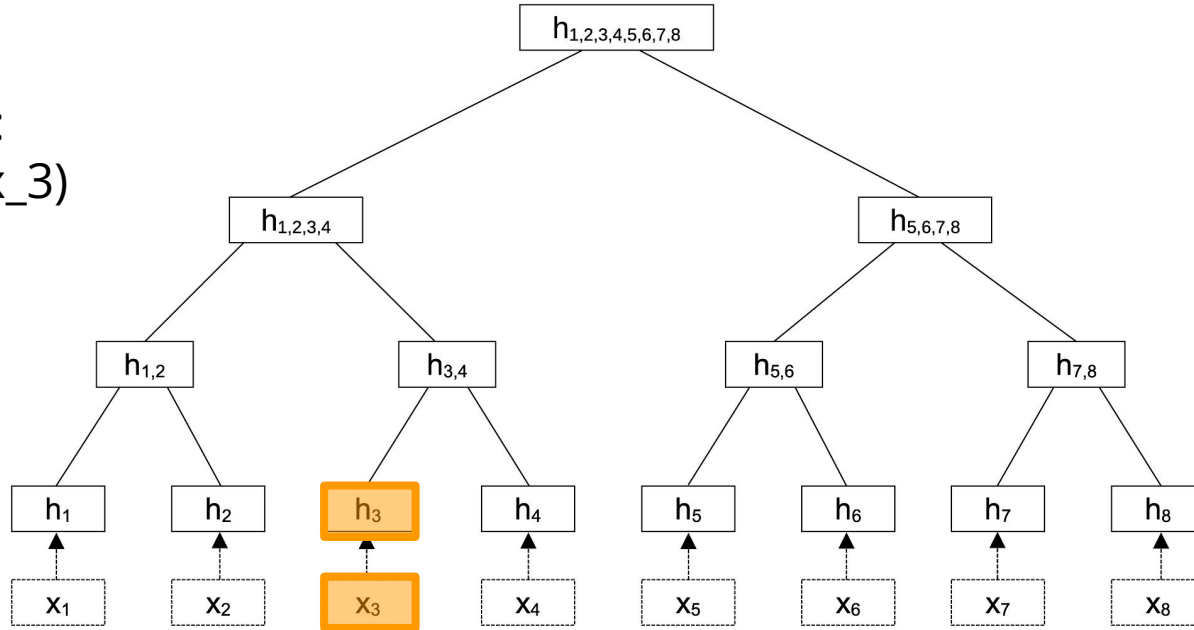
Merkle tree



Merkle tree

The i -th leaf node is the cryptographic hash of the i -th data object.

Example:
 $h_3 = H(x_3)$

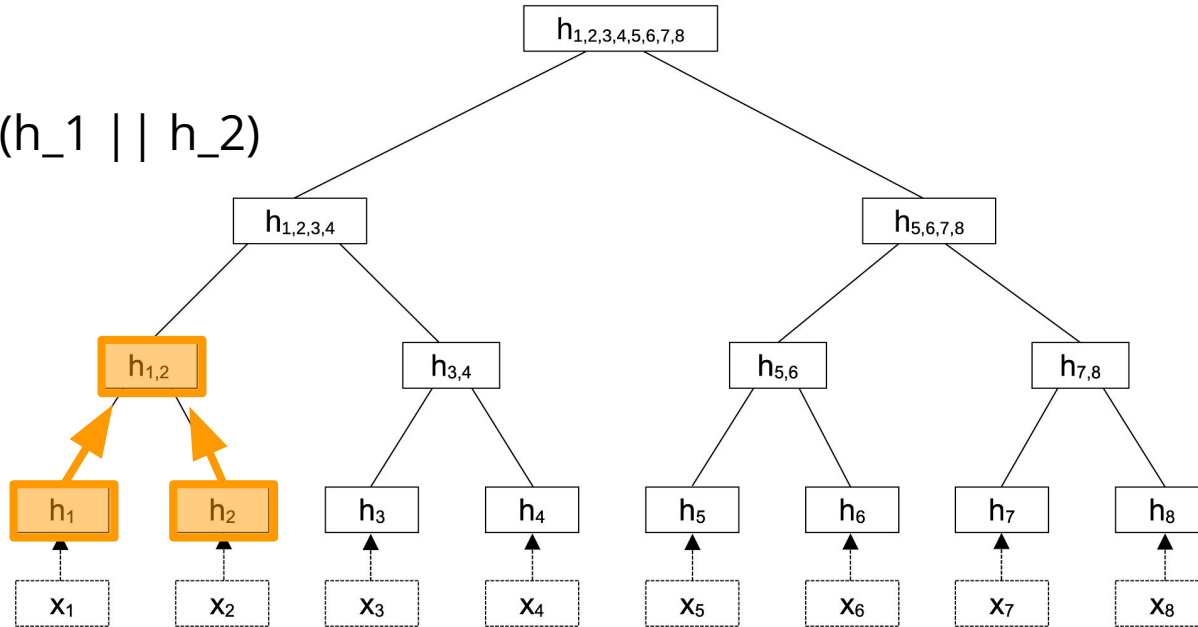


Merkle tree

Internal nodes are labeled by hashing the digests of their children.

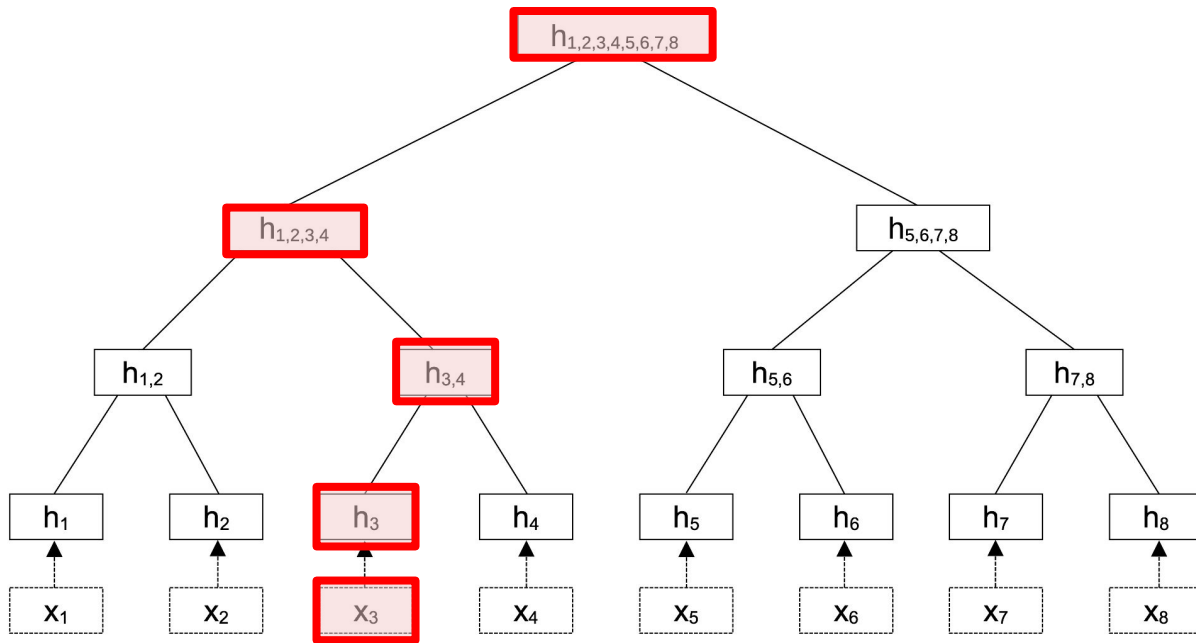
Example:

$$h_{\{1,2\}} = H(h_1 \parallel h_2)$$



Merkle tree

Modifying a leaf causes a change in the root of the tree (and hence in the global digest).

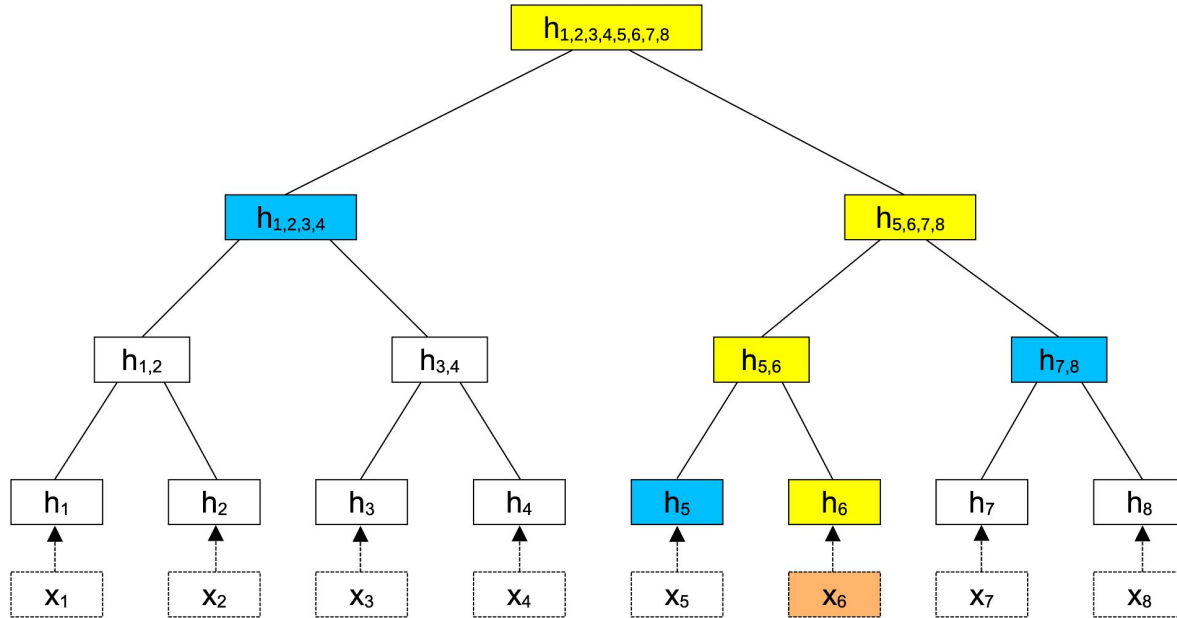


Merkle tree

To modify a leaf of the Merkle tree while keeping the root unchanged, a malicious user would need to find a collision for the hash function used.

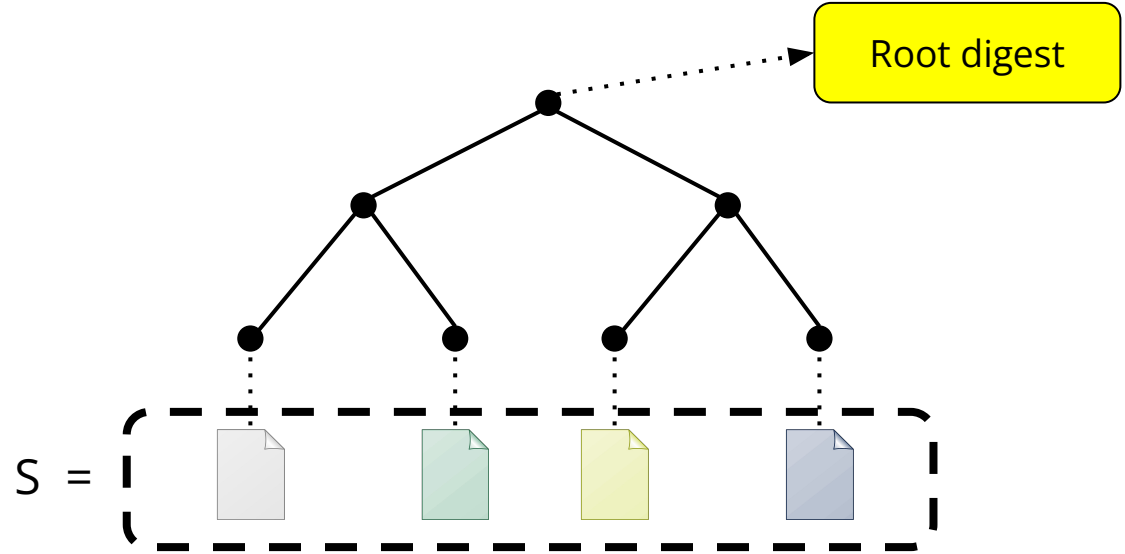
Merkle tree

Membership proof: certifies that an element x is included in the set S .



Proof

Merkle tree

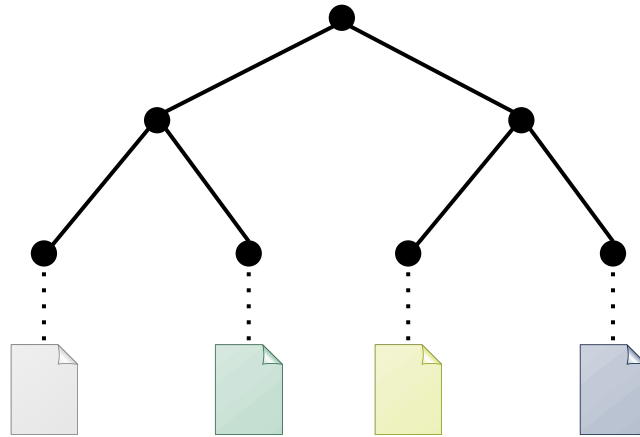


Prover

The prover builds the tree for S and publishes the resulting root digest.

Merkle tree

Root digest



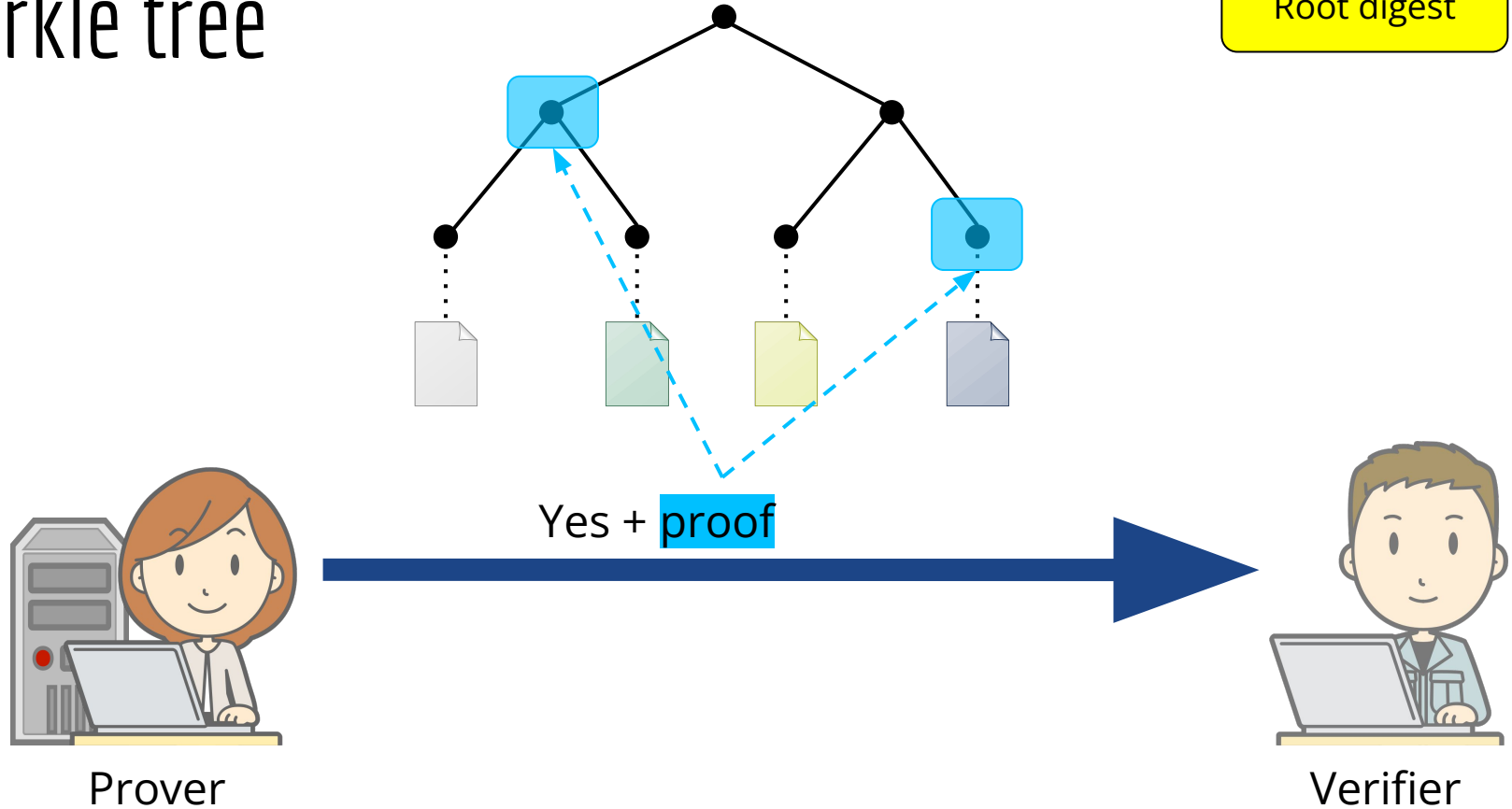
Prover

Is  in the set?

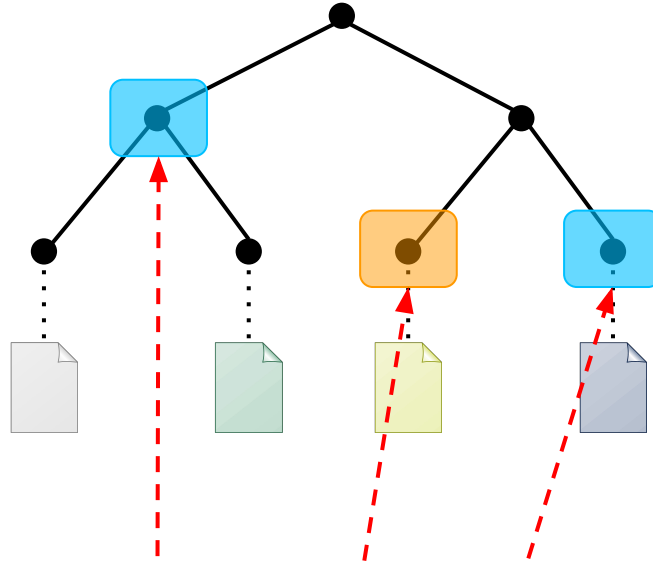


Verifier

Merkle tree



Merkle tree



Root digest

The verifier uses these values to reconstruct the tree root.

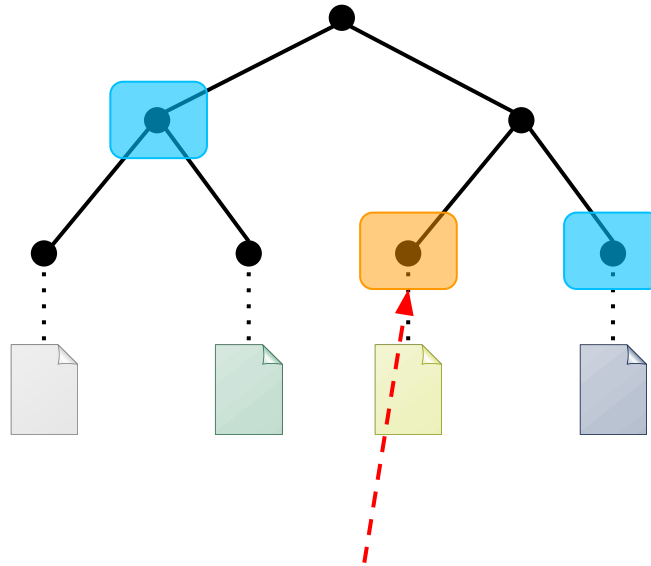


Prover



Verifier

Merkle tree



Root digest



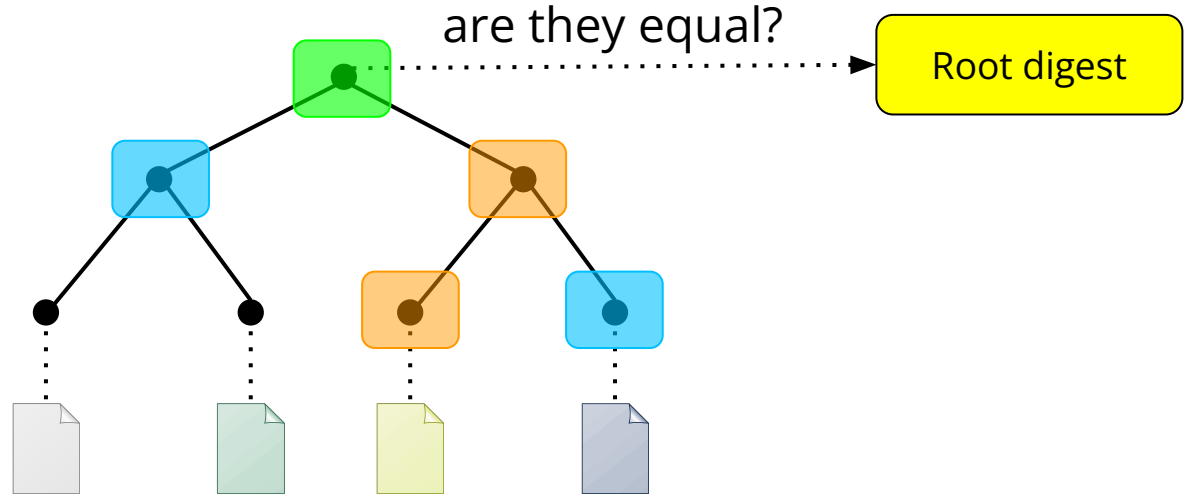
Prover

The verifier already knows this as it represents the file hash.



Verifier

Merkle tree



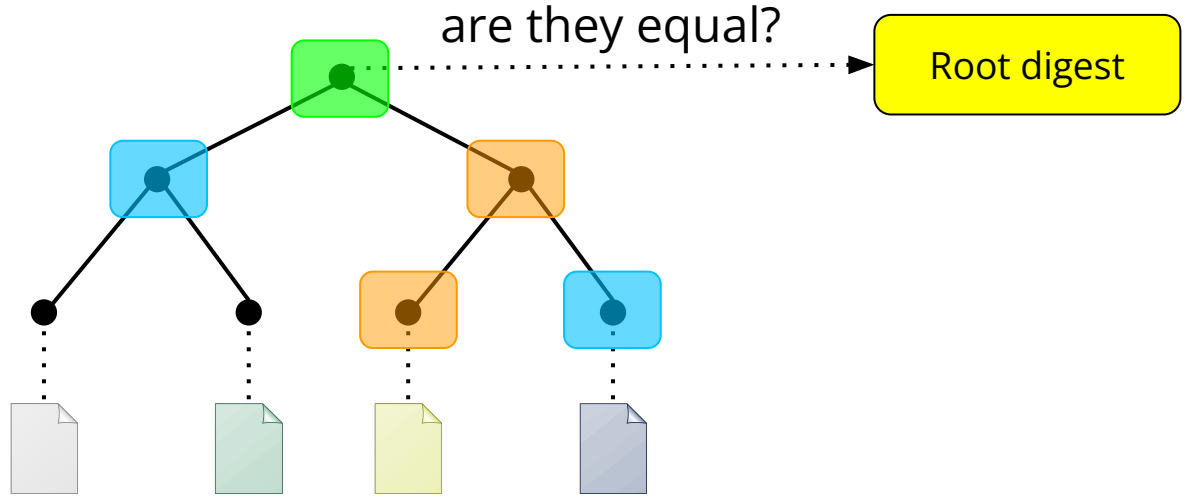
Prover

The **reconstructed root** is compared with the **public root digest**.



Verifier

Merkle tree



Prover

If they coincide, the proof is accepted as correct.



Verifier



Hands-on session



Thank you
for your attention

